PROJECT THEORY GUIDE

RISK ALERT CLASSIFIER



Simple English • Formulas • Worked examples

Understand how customer information becomes a risk warning—and why catching the risky minority matters more than accuracy alone.

4,600	16	2
CUSTOMER RECORDS	INPUT FIELDS	RISK CLASSES

Prepared for the DevanshiCodesAI / Risk_Classifier project

WHAT YOU WILL LEARN

- How data is prepared and missing values are filled
- How classifiers learn and how their errors are measured
- Why balancing, tuning, and thresholds change the trade-offs
- How to connect model scores to responsible business decisions

Educational guide. Worked examples are illustrative unless marked as saved outputs.

01 / PROJECT OVERVIEW

The project in everyday language

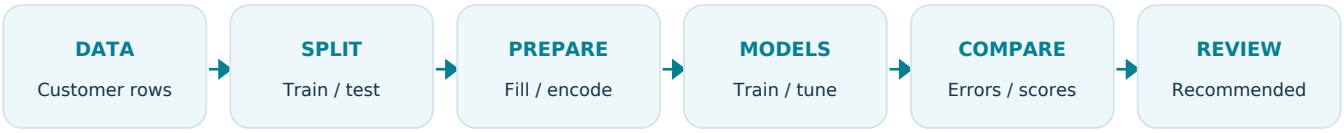
Think of this project as a **smoke alarm for customer risk**. It studies past examples, looks for patterns, and raises a warning for customers that resemble the high-risk group. A warning is a reason to investigate—not proof of fraud or future default.

This is **supervised binary classification**: the training examples already have an answer, and the model learns to predict one of two labels. **0 = Low Risk; 1 = High Risk.** High Risk is the positive class throughout this guide.

Project fact	What it means
4,600 records; 19 columns	Each row represents a customer.
16 input fields before encoding	13 numerical fields and 3 categorical fields.
Target: risk_status	The answer the model learns to predict.
4,043 low risk; 557 high risk	87.89% low risk and 12.11% high risk.
80/20 split	3,680 training customers; 920 test customers.

Input examples: income, credit score, missed payments, late-payment days, spending, account tenure, gender, region, and employment type. The customer ID and raw transaction date are excluded.

The learning workflow



Human review is a recommended use after validation; the repository is a learning notebook, not a deployed banking service.

Reading map

Pages 3–9: learn the basics	Pages 10–16: compare and apply
3 · Data preparation	10 · Decision Trees
4 · Logistic Regression	11 · Random Forest and overfitting
5 · Confusion matrix and error types	12 · Tuning and cross-validation
6 · Metric formulas	13 · Business costs and thresholds
7 · ROC, AUC, and precision–recall	14 · Your notebook: code and checks
8 · Class imbalance; under/over-sampling	15 · Revision and formula sheet
9 · SMOTE and ADASYN	16 · Submission checklist and references

02 / INPUTS, MISSING VALUES, AND LEAKAGE

Prepare the data fairly

X is the customer information; **y** is the known risk label. Remove the answer from X so the model cannot simply read it. Split the rows **before learning any filling or scaling rules**.

1. Keep training and testing separate

Training is practice; testing is an unseen exam. **Stratification** keeps roughly the same class proportions in both groups. The current notebook uses an 80/20 random split with seed 42, but does not use `stratify=y`; adding it is a recommended improvement.

2. Put numerical columns on comparable scales

STANDARD SCALING

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

x = original value; **μ** = training-column mean; **σ** = training-column standard deviation. Example: age 50, mean 40, standard deviation 10 → scaled age = 1.

Scaling stops a large rupee amount from dominating an age value when comparing customers. It does not mean every value is between 0 and 1. Learn μ and σ on training data only.

3. Fill missing numbers with KNN imputation

SIMPLE UNIFORM-NEIGHBOR AVERAGE

$$\hat{x} = \frac{x_1 + x_2 + \cdots + x_k}{k}$$

x̂ = filled value; **$x_1 \ldots x_k$** = the available values from similar training customers; **k** = number used, up to 5 here. Similarity uses available, scaled numerical information.

Illustrative credit scores from five neighbors: 600, 620, 640, 660, 680. Their average is **640**. KNN estimates a missing value; it does not discover its true value. This imputer is **not** a KNN classification model and does not need the risk label.

4. Handle categories

Fill missing categories using the training set's most common category. **One-hot encoding** turns categories into separate yes/no columns: for example, North, South, West become [1,0,0], [0,1,0], [0,0,1]. These are example categories, not an ordering from best to worst.

WHY "TRAIN ONLY" MATTERS

Learning preprocessing rules from the test set is data leakage: the unseen exam has helped prepare the student. Use the same principle inside every cross-validation fold.

03 / A SIMPLE CLASSIFICATION BASELINE

Logistic Regression: a risk probability

Despite its name, Logistic Regression is suitable for classification. It combines customer clues into a score, then maps that score to a probability between 0 and 1. The model learns its weights from labeled training examples.

STEP 1 — COMBINE THE CLUES

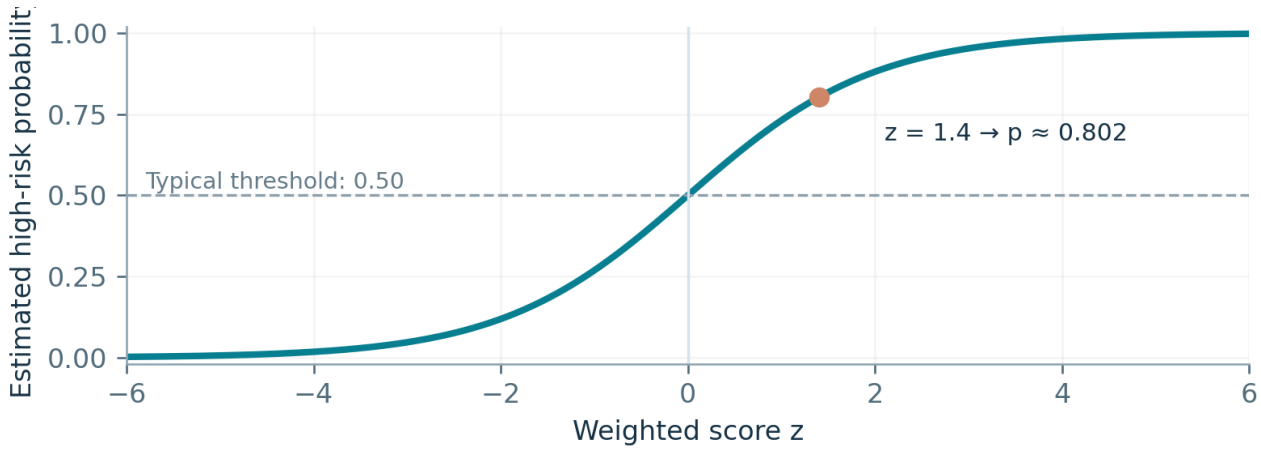
$$z = b + \sum_{j=1}^{d} w_j x_j$$

z = combined score; **b** = starting offset; **d** = number of model inputs; $\mathbf{x_j}$ = input value; $\mathbf{w_j}$ = learned weight. Positive weights raise z; negative weights lower it.

STEP 2 — THE SIGMOID FUNCTION

$$p = P(y = 1 \mid x) = \frac{1}{1 + e^{-z}}$$

p = estimated High Risk probability; **e** ≈ 2.718, the exponential constant. The S-shaped curve keeps the result between 0 and 1.



Illustrative sigmoid curve; it is a mathematical function, not a fitted model result.

Worked example: hand-picked weights, not notebook coefficients

Suppose z = −2 + 0.8m + 0.1L, where **m** is missed payments and **L** is average late-payment days. For m = 3 and L = 10: z = −2 + 2.4 + 1 = **1.4**, so p ≈ **0.802**.

A conventional decision rule is: flag High Risk when **p ≥ t**, where **t** is the threshold. With t = 0.50, this example is flagged. A lower threshold usually catches more risks but also creates more false alerts.

In the notebook: LogisticRegression(max_iter=2000). “2000” is an optimization-iteration limit, not a number of customers or trees. An estimated probability is not automatically a well-calibrated real-world default probability.

Calibration means the probabilities match observed frequencies: among many comparable customers given a risk probability of 0.20, about 20% should have the High Risk label. This must be checked, not assumed.

04 / CONFUSION MATRIX AND ERROR TYPES

Understand the four possible outcomes

ILLUSTRATIVE EXAMPLE — NOT NOTEBOOK RESULTS

Imagine 1,000 test customers: 120 truly high risk and 880 truly low risk. The model flags 150 customers. Of those alerts, 90 are correct and 60 are unnecessary.

A **confusion matrix** is a scorecard of correct and incorrect decisions. Rows below are the true labels; columns are the predicted labels.

True label ↓ / Predicted →	Low Risk (0)	High Risk (1)
Low Risk (0)	**TN = 820** Correctly not flagged	**FP = 60** Unnecessary alerts
High Risk (1)	**FN = 30** Missed warnings	**TP = 90** Correct risk alerts

Check the totals: actual high risk = TP + FN = **120**; actual low risk = TN + FP = **880**; predicted alerts = TP + FP = **150**; all customers = **1,000**.

Type I error: False Positive (FP)

The model flags a low-risk customer as high risk. This can create extra verification, manual review, inconvenience, or an unfair decision if the flag is treated as final. In the example there are **60 false alerts**.

Type II error: False Negative (FN)

The model misses a genuinely high-risk customer. A bank may lose the opportunity to investigate or manage exposure. In the example there are **30 missed warnings**. This project places particular importance on reducing these misses.

Why the positive class matters

“Positive” means **class 1 — High Risk**, not “good.” Reversing the positive class changes the meaning of precision and recall. Keep the class definition and row/column labels clear.

BUSINESS INTERPRETATION

False positives and false negatives have different consequences. The actual costs depend on the bank and the action taken after an alert; they are not established by this dataset.

05 / ONE EXAMPLE, SEVERAL USEFUL QUESTIONS

The important classification formulas

Use the same illustrative counts: **TP = 90, TN = 820, FP = 60, FN = 30**. Here N = TP + TN + FP + FN = 1,000. Scores below lie between 0 and 1; multiplying by 100 gives a percentage.

Metric and question	Formula	Worked result
Accuracy How many labels were correct?	(TP + TN) / N	910 / 1000 **0.9100 = 91%**
Precision How trustworthy were the alerts?	TP / (TP + FP)	90 / 150 **0.6000 = 60%**
Recall = TPR How many actual risks were caught?	TP / (TP + FN)	90 / 120 **0.7500 = 75%**
F1-score How do precision and recall balance?	2PR / (P + R) = 2TP / (2TP + FP + FN)	180 / 270 **0.6667**
FPR How many low-risk customers were falsely flagged?	FP / (FP + TN)	60 / 880 **0.0682 = 6.82%**
Specificity = TNR How many low-risk customers were correctly left unflagged?	TN / (TN + FP) = 1 − FPR	820 / 880 **0.9318 = 93.18%**
FNR What share of actual risks was missed?	FN / (FN + TP) = 1 − Recall	30 / 120 **0.2500 = 25%**
Balanced accuracy How well was each class detected, giving equal weight to both?	(Recall + Specificity) / 2	(0.7500 + 0.9318) / 2 **0.8409 = 84.09%**

P = precision; **R** = recall; **N** = number of evaluated customers. TP, TN, FP, FN are counts, not probabilities.

Precision and recall are different

Precision starts with **our alerts**. Recall starts with **the truly risky customers**. Increasing sensitivity can improve recall while lowering precision. F1 uses a harmonic mean, so a very low precision or recall pulls the score down.

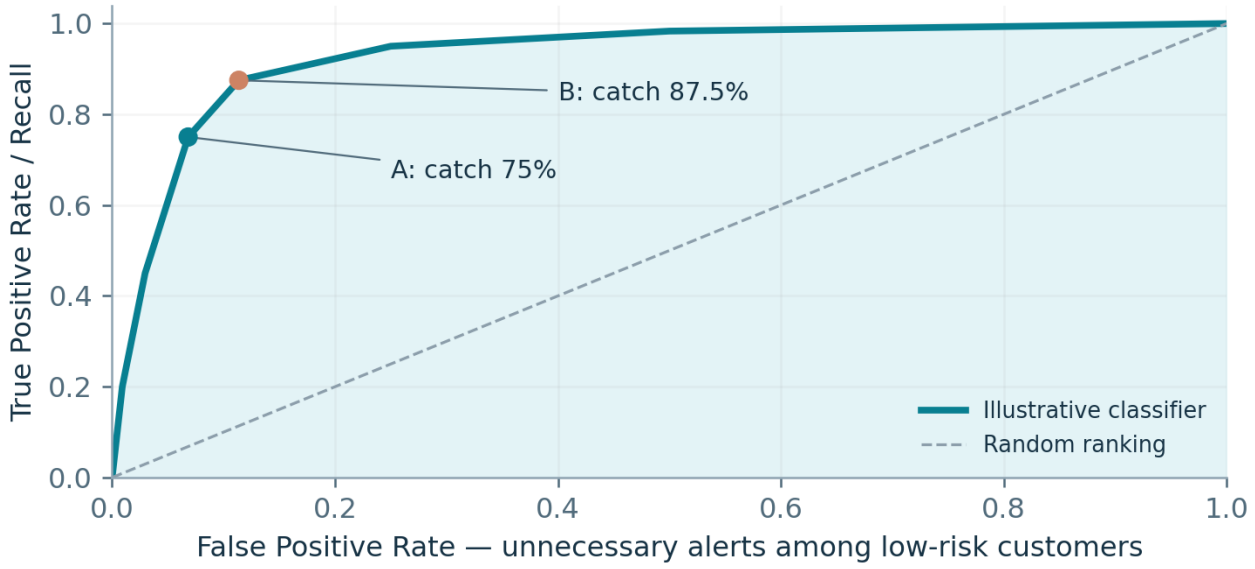
A ZERO DENOMINATOR

If no customers are flagged, precision has a zero denominator. If there are no actual positives, recall is undefined. Choose and report the software's zero-division behavior; do not silently treat "undefined" as a meaningful success.

06 / EVALUATE SCORES, NOT ONLY LABELS

ROC, AUC, and the threshold trade-off

A **ROC curve** plots **FPR on the horizontal axis** and **TPR/recall on the vertical axis** while the threshold changes. Near the top-left is desirable: catch many risks while falsely flagging few low-risk customers.



Illustrative ROC curve. A and B use the teaching example, not the repository's model predictions.

AREA UNDER THE ROC CURVE

$$\mathrm{AUC} = \int_0^1 \mathrm{TPR}(u)\,du$$

u is the false-positive rate. AUC summarizes the area under the ROC curve. Software usually estimates the area from evaluated points; it is not accuracy at one threshold.

AUC ≈ 0.5: roughly random ranking. **AUC = 1:** perfect ranking on the evaluated data. For example, AUC 0.90 means a randomly chosen high-risk customer is scored above a randomly chosen low-risk customer about 90% of the time, counting ties as half.

Lower thresholds create a business trade-off

Example A catches 90 of 120 risks, with 60 false alerts. A more sensitive setting B catches 105 risks, with 100 false alerts. Recall rises from **75% to 87.5%**, but the review team has more alerts to handle.

Also look at precision–recall

For a minority class, a precision–recall curve and **average precision** help show whether extra alerts are useful. A small FPR can still produce many false alerts when low-risk customers are numerous.

Correct ROC input: each model's own High Risk probabilities, such as predict_proba(test_data)[:, 1]. Reusing one model's probabilities for every curve gives a misleading comparison.

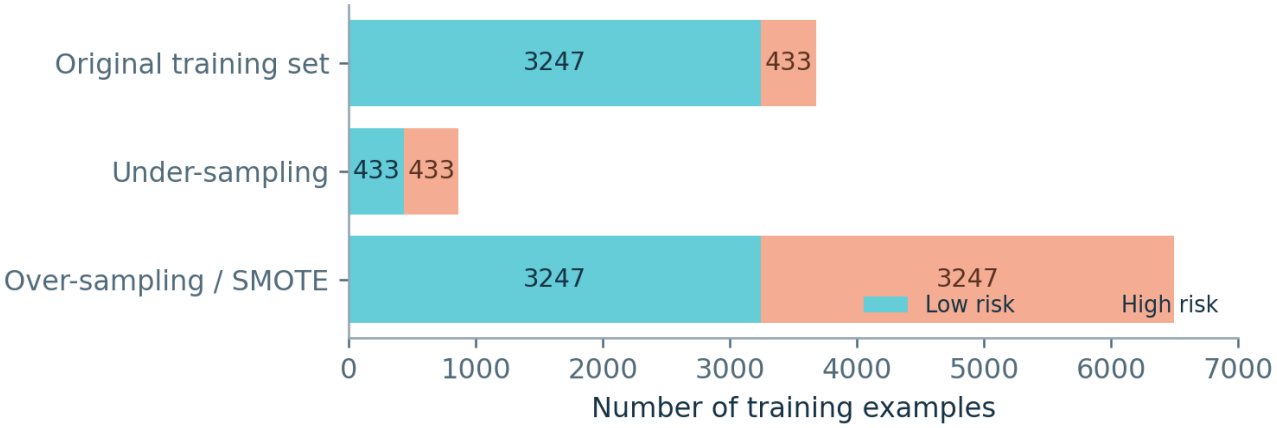
07 / UNDER-SAMPLING AND OVER-SAMPLING

Why class imbalance changes the problem

In the full project dataset, **87.89%** of customers are low risk and **12.11%** are high risk. Calling everyone low risk would score 87.89% accuracy but **0% high-risk recall**. That illustrates why accuracy alone can fail the business goal; it is not the saved Logistic Regression result.

Balance the training examples—not the test customers

The current saved training split contains **3,247 low-risk** and **433 high-risk** customers. Default binary balancing would give the following sample sizes. These are derived sizes, not a new model run.



Expected default training counts: under-sampling keeps 433 per class; over-sampling/SMOTE target 3,247 per class. ADASYN can produce approximate counts.

Method	What changes	Main caution
Random under-sampling	Keep fewer majority examples; here discard 2,814 low-risk examples.	Less training information; some useful patterns may be lost.
Random over-sampling	Repeat minority examples; here add 2,814 high-risk copies.	Repeated examples do not create new independent information.
SMOTE / ADASYN	Create synthetic minority training examples.	Artificial cases can add noise or unrealistic combinations.

Sampling changes what the model sees during training; it does not change how common risky customers are in real life. Keep the test set at its original class mix. Compare recall, precision, F1, ROC-AUC, and review volume afterward.

WHEN TO RESAMPLE

Inside cross-validation, resample only the training portion of each fold. Otherwise copied or synthetic information can leak into validation. Balancing is not guaranteed to improve performance, and scores may need probability calibration afterward.

08 / SYNTHETIC MINORITY SAMPLING

How SMOTE and ADASYN create examples

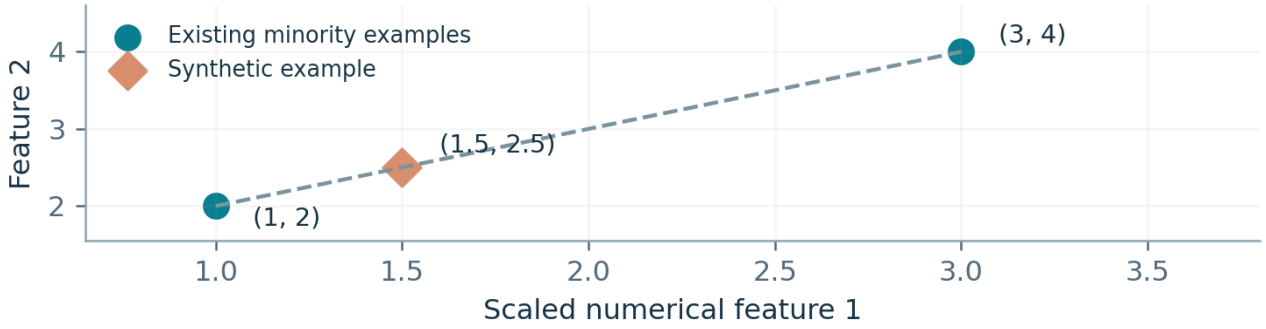
SMOTE: make a point between two minority examples

SMOTE INTERPOLATION

$$x_{\text{new}} = x_i + \lambda(x_j - x_i), \quad 0 \le \lambda \le 1$$

$\mathbf{x_i}$ = a minority example; $\mathbf{x_j}$ = a nearby minority example; $\boldsymbol{\lambda}$ = a random fraction. Inputs can be numerical vectors; the new example receives the minority label.

Illustrative example: $x_i = (1, 2)$, $x_j = (3, 4)$, $\lambda = 0.25$. Then $x_{new} = (1, 2) + 0.25(2, 2) =$ **(1.5, 2.5)**. These are example scaled features, not actual customer records.



SMOTE interpolates; it does not invent a verified real customer. The notebook applies it to 13 scaled numerical fields.

ADASYN: spend more examples on harder regions

DIFFICULTY AND SAMPLE ALLOCATION

$$r_i = \frac{\Delta_i}{k}, \qquad g_i \approx G\frac{r_i}{\sum_j r_j}$$

$\mathbf{k}$ = number of nearby examples examined; $\boldsymbol{\Delta_i}$ = how many are majority-class examples; $\mathbf{r_i}$ = local difficulty; $\mathbf{G}$ = total new examples requested; $\mathbf{g_i}$ = approximate number allocated near point i. The sum adds difficulty across all minority points; rounding can change the final total.

Example: three minority points have 1, 3, and 5 majority neighbors out of 5. Their difficulty ratios are 0.2, 0.6, and 1.0. With a budget of 9 new examples, the shares are about **1, 3, and 5**. The hardest region gets more examples.

THE LIMITATION

Hard cases may also be outliers or incorrectly labeled records. ADASYN can amplify them. If all difficulty ratios are zero, the usual allocation formula cannot be normalized. Compare synthetic sampling against simpler methods on fair validation data.

09 / GINI, ENTROPY, AND USEFUL SPLITS

Decision Trees: questions that separate groups

A Decision Tree learns a sequence of questions, such as whether a numerical value is above a threshold. Each split aims to create groups with more consistent labels. Final groups are called **leaves**. The questions are learned from training data; they are not manually assumed risk rules.

GINI IMPURITY

$$\text{Gini} = 1 - \sum_{c=1}^{C} p_c^2$$

C = number of classes (2 here); $\mathbf{p_c}$ = the fraction of records in class c within a node. Pure nodes have Gini 0; an evenly mixed binary node has Gini 0.5.

ENTROPY

$$H = -\sum_{c=1}^{C} p_c \log_2(p_c)$$

H measures label uncertainty; $\log_2$ is a base-2 logarithm. Treat $0 \times \log_2(0)$ as 0 by its limiting value. Pure nodes have entropy 0; a 50/50 binary node has entropy 1.

SPLIT GAIN — WEIGHT THE CHILD GROUPS

$$\text{Gain} = I_P - \frac{n_L}{n} I_L - \frac{n_R}{n} I_R$$

I = chosen impurity measure; subscripts P/L/R mean parent/left/right; **n** = parent count; **nL, nR** = child counts. Bigger impurity reduction is better, subject to the tree settings.

Worked Gini example

A parent has 80 low-risk and 20 high-risk records. Its Gini is $1 - 0.8^2 - 0.2^2 =$ **0.32**. A possible split creates:

Group	Counts	Gini
Left child: 60 records	60 low risk; 0 high risk	0
Right child: 40 records	20 low risk; 20 high risk	0.50

Weighted child impurity = (60/100)×0 + (40/100)×0.50 = **0.20**. Gain = 0.32 − 0.20 = **0.12**. This is an illustrative split, not a reported notebook tree.

Deeper trees can memorize noise. max_depth, min_samples_split, and min_samples_leaf limit complexity. In the notebook, the criterion search compares gini and entropy.

10 / MANY TREES, A STEADIER PREDICTION

Random Forest and generalization

Random Forest trains many trees using bootstrap samples of the training rows and random subsets of features at splits. **Bootstrap** means sampling with replacement, so a row may appear more than once in a tree's training sample. These differences make the trees less identical.

AVERAGE THE TREE PROBABILITIES

$$p_{\mathrm{RF}}(y=1 \mid x) = \frac{1}{B}\sum_{b=1}^{B} p_b(y=1 \mid x)$$

B = number of trees; $\mathbf{p_b}$ = the High Risk probability from tree b. In scikit-learn, the forest averages tree probabilities; calling this "voting" is a useful simplified description.

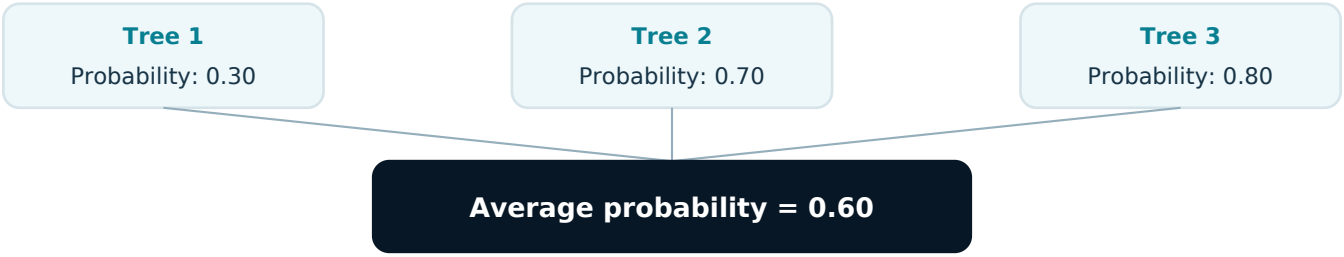


Illustration with three trees: (0.30 + 0.70 + 0.80) / 3 = 0.60. The notebook starts with 100 trees before tuning.

Overfitting versus underfitting

Overfitting: the model memorizes details that do not transfer well to new customers.
Underfitting: the model is too simple to learn useful patterns even from training data. Check training and validation/test performance, not just the training score.

A SIMPLE TRAIN/TEST GAP

$$\mathrm{Gap} = \mathrm{Accuracy}_{\mathrm{train}} - \mathrm{Accuracy}_{\mathrm{test}}$$

A large positive gap is a warning sign, not proof by itself. Also check minority recall and precision; accuracy can hide class-specific problems.

Saved notebook output	Train accuracy	Test accuracy	Gap
Decision Tree	1.0000	0.9620	0.0380
Random Forest	1.0000	0.9880	0.0120

These are stored notebook results, not a fresh execution. The forest shows a smaller accuracy gap on that split, but independent validation is still needed.

NO AUTOMATIC GUARANTEE

More trees can stabilize an ensemble; they do not fix bad labels, data leakage, unfair inputs, or a changing customer population. A strong test score is not proof that a model is ready for real banking use.

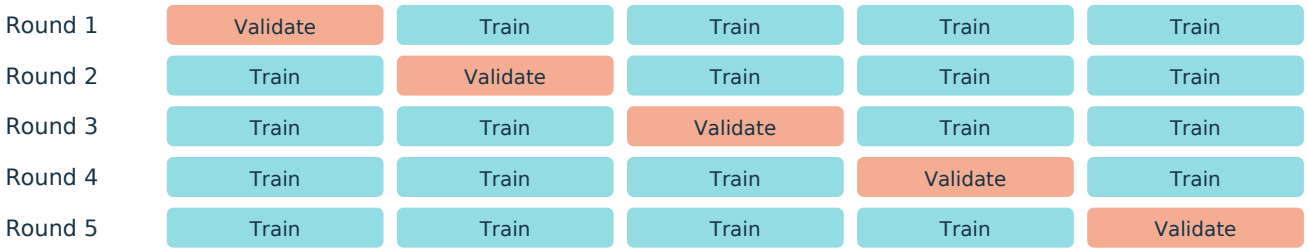
11 / RANDOMIZED SEARCH AND GRID SEARCH

Tune settings with cross-validation

Model parameters are learned from examples, such as Logistic Regression weights. **Hyperparameters** are settings chosen before training, such as tree depth or number of trees. Tuning compares these settings using validation data.

Five-fold cross-validation

Divide the training data into five folds. Train on four and validate on the fifth, then rotate. For classification, stratified folds help preserve class proportions. Learn preprocessing and apply sampling separately inside each fold's training portion.



Five folds of the training data; the final test set stays separate.

AVERAGE VALIDATION RECALL

$$\overline{R}_{CV} = \frac{1}{K}\sum_{k=1}^{K} R_k$$

K = number of folds (5); $\mathbf{R_k}$ = High Risk recall on validation fold k. The notebook uses scoring="recall" because catching high-risk customers is its priority.

Search type	What it tries	Project use
RandomizedSearchCV	Selected combinations from the search space.	20 combinations each for Decision Tree and Random Forest.
GridSearchCV	Every combination in a defined grid.	Fine-tunes Random Forest with 36 combinations.

A randomized search uses 20 × 5 = **100 validation fits**, plus a best-model refit. The forest grid has 3 tree-count choices × 3 depth choices × 2 split choices × 2 leaf choices = **36 combinations**; 36 × 5 = **180 validation fits**, plus a refit.

Main settings: n_estimators = trees; max_depth = maximum levels; min_samples_split = records needed to split; min_samples_leaf = minimum leaf size; criterion = impurity measure. random_state makes randomness repeatable; n_jobs=-1 uses available CPU cores.

KEEP THE FINAL TEST SET OUT OF TUNING

Choose settings and thresholds with training/validation data. Evaluate the final choice on the untouched test set. Repeatedly choosing a winner from the same test set makes the final estimate less trustworthy.

12 / THRESHOLDS, WORKLOAD, AND RESPONSIBLE USE

Connect the model to business costs

The best model is not necessarily the most accurate one. It should reduce costly missed warnings without overwhelming the review team or unfairly affecting customers. Recall alone is not enough: flagging everyone would catch every risk but create many unnecessary alerts.

A SIMPLIFIED ERROR-COST MODEL

$$\text{Cost} = C_{FN}\,FN + C_{FP}\,FP$$

C_FN = assumed cost per missed risk; **C_FP** = assumed cost per false alert. Multiply each cost by its error count. This simplified model omits other real business costs and benefits.

ILLUSTRATIVE COSTS — NOT BANK ESTIMATES

Assume a missed risk costs ₹10,000 and a false alert costs ₹500. Use the same 1,000 teaching customers. Setting B is more sensitive than A; these are hypothetical operating points.

Measure	Setting A	Setting B
Correct risk alerts: TP	90	105
Missed risks: FN	30	15
False alerts: FP	60	100
High-risk recall	90/120 = 75%	105/120 = 87.5%
Alert precision	90/150 = 60%	105/205 ≈ 51.22%
Total accounts to review	150	205
Overall accuracy	910/1000 = 91%	885/1000 = 88.5%
Simplified error cost	30×10,000 + 60×500 **₹3,30,000**	15×10,000 + 100×500 **₹2,00,000**

Setting B has **lower accuracy but lower assumed error cost**. It also produces 55 more reviews and lower alert precision. Whether that is acceptable depends on staffing, actual costs, fairness, and reliable validation.

A responsible decision process

- Select thresholds using validation data, not the final test results.
- Use a risk flag to trigger human review, not automatic denial based only on a label.
- Check performance across customer groups; review the use of age, gender, region, and other sensitive or proxy information.
- Protect customer data, check that inputs exist before the decision, and monitor performance as customer behavior changes.

13 / LINK THE THEORY TO THE IMPLEMENTATION

What your notebook does—and what to check

Code / object	Simple meaning
`fit(...)`	Learn from training data.
`transform(...)`	Apply already-learned preparation rules.
`predict(...)`	Return a risk label: 0 or 1.
`predict_proba(...)[:, 1]`	Return estimated probabilities for High Risk.
`ColumnTransformer`	Prepare numerical and categorical fields differently.
`Pipeline / ImbPipeline`	Connect preparation, optional training sampling, and the model.

Two feature paths in the current code

- **All 16 input fields:** baseline and under-sampling Logistic Regression use numerical scaling plus one-hot encoded categories.
- **13 scaled numerical fields:** over-sampling, SMOTE, ADASYN Logistic Regression, Decision Tree, and Random Forest.

Because the inputs differ, a performance change cannot be credited to sampling alone. A fair experiment should keep the features and preprocessing consistent, changing only the technique being compared.

Important checks before submitting final conclusions

- **Split:** add stratify=y if maintaining class proportions is intended; rerun all later cells because the split will change.
- **ROC/AUC:** the current loops reuse y_prob. Calculate each estimator’s own probabilities with its matching input representation.
- **Cross-validation:** numerical scaling and KNN filling occur before the tree searches. Move them inside the searched pipeline so each fold fits its own rules.
- **Report mismatch:** Part H quotes under-sampling recall 0.4919, while its saved output shows 1.0000. Stored confusion matrices show 0 missed risks for both baseline and under-sampling, with 1 versus 8 false alerts. Refresh the narrative after a clean run.
- **Model choice:** under-sampling is selected manually. The saved outputs do not justify calling it uniquely best. Recheck high scores for leakage and validate on independent data.

SCOPE OF THIS PDF

This guide explains theory and inspects the existing notebook snapshot. It does not modify or rerun the notebook, verify fraud/default outcomes, or certify a production-ready banking model.

14 / A COMPACT STUDY SHEET

Quick revision: concepts and formulas

Concept	Formula or key relationship
Standard scaling	$x_{scaled} = (x - \mu) / \sigma$
Logistic Regression	$z = b + \Sigma w_j x_j$; $p = 1 / (1 + e^{-z})$
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$
Precision / Recall	Precision = TP/(TP+FP) Recall = TP/(TP+FN)
F1-score	2PR/(P+R) = 2TP/(2TP+FP+FN)
FPR / FNR	FPR = FP/(FP+TN) FNR = FN/(FN+TP)
Gini / Entropy	Gini = $1 - \Sigma p_c^2$ Entropy = $-\Sigma p_c \log_2(p_c)$
SMOTE	$x_{new} = x_i + \lambda(x_j - x_i)$, $0 \le \lambda \le 1$
Cross-validation / Cost	CV = mean fold score Cost = C_FN×FN + C_FP×FP

μ = training mean; σ = training standard deviation; P/R = precision/recall; λ = interpolation fraction; p_c = class share in a tree node. “Σ” means add the listed terms.

Six short theory answers

Why Logistic Regression for classification? It estimates a probability between 0 and 1 using a sigmoid. A threshold turns that probability into a class label.

Why is accuracy alone insufficient? The majority class can dominate the score. A model may be highly accurate while missing every high-risk customer.

What are Type I and Type II errors? Type I is a false risk alert (FP). Type II is a missed actual risk (FN).

How do precision and recall differ? Precision asks how many alerts were correct. Recall asks how many actual risks were caught. TPR equals recall; FPR concerns low-risk customers.

What does ROC-AUC measure? How well risk scores rank the two classes across thresholds. It is not accuracy or proof of real-world safety.

Why is imbalance difficult? The learner may favor common low-risk examples. Sampling can change training attention, but improvements must be measured on unchanged evaluation data.

THE CORE IDEA
Build a useful warning system, not just an impressive score. Catch important risks, understand false alerts, and validate decisions fairly.

15 / FINISH WITH CLEAR, HONEST DOCUMENTATION

Submission checklist and references

Submit the theory guide alongside the notebook, dataset documentation, evaluation tables/plots, and README. Clearly separate **teaching examples**, **saved outputs**, and **newly reproduced results**.

Checklist for a strong project submission

- State the positive class: High Risk = 1. Explain the business goal and the limits of the label.
- Document feature selection, missing-value handling, scaling, encoding, and the train/test split.
- Compare models fairly with the same information; perform preprocessing and sampling inside CV training folds.
- Report confusion matrices, accuracy, precision, minority recall, F1, and model-specific ROC-AUC.
- Label plots and add a short plain-English interpretation beneath each one.
- Select model settings and the operating threshold with validation data; reserve the final test set for evaluation.
- Rerun the notebook in order, correct the ROC loops, and align the written report with the generated results.
- Discuss review workload, error costs, privacy, group fairness, and independent validation.

Sources and further reading

Project repository and notebook
Source of dataset facts, code structure, and saved outputs used in this guide.

scikit-learn: Logistic Regression
Classification probabilities, model fitting, and regularization.

scikit-learn: classification metrics
Precision, recall, F1, confusion matrices, ROC-AUC, and related metrics.

scikit-learn: trees, forests, and model selection
Decision Trees, Random Forest, preprocessing, cross-validation, and parameter search.

imbalanced-learn: sampling methods
Random sampling, SMOTE, ADASYN, and pipelines for imbalanced learning.

DOCUMENTATION NOTE
Dataset counts were checked against the repository CSV. Stored metrics were read from the notebook; no models were retrained for this PDF. All hypothetical probabilities, matrices, synthetic points, and business costs are explicitly labeled as illustrative.